Secure Embedded Water Tank Controller

Finley Burns
CYCS 3240 Testing Design
Due November 12, 2026

What I am testing

I am testing whether my C17 controller makes the right valve decisions, rejects invalid input, and keeps controlling the tank while it handles operator commands. The project still has one simulated tank, one level sensor, one inlet valve, and a serial interface on an emulated STM32F4 Cortex-M4. This document explains the tests, the settings I will use, and the results I expect.

I will read the test data from files so I can repeat each test with the same inputs. I will test the shared C control function on my computer, then check the firmware and its interfaces in Renode. After that, I will connect the changing tank model. Renode can run automated Robot Framework tests and save their results [1].

What works now

The current prototype checks the level and settings, uses hysteresis to control the valve, and closes the inlet on invalid input. The firmware still uses one fixed 25% reading. For this submission, I added tests around the thresholds, sequences that keep the previous state, and invalid input files. I ran these tests on Windows. I still need to add changing sensor input, periodic sampling, the tank model, serial parsing, and HMAC authentication. The tests for those features describe what I will do once they are implemented.

Starting parameters

Level and thresholds. I use 0 to 10000 basis points, where one unit is 0.01% of capacity. The low threshold is 3000 and the high threshold is 7000. Integer values make the endpoint checks exact, and the gap makes it easy to follow the valve decisions.

Sampling. I will start with one reading every 100 ms of simulated time. I will also test 50 and 500 ms to see how the time between readings affects the response.

Tank and flow. I will use a 100 L tank starting at 50 L, with 2 L/s entering when the valve is open and 1 L/s leaving. These are simple simulation values chosen to produce repeated filling and draining.

Run length. The normal test will last 600 simulated seconds and run three times. That is long enough for several fill and drain cycles and lets me check that repeated runs agree.

I kept the 30% and 70% thresholds and 100 ms period from the preliminary design. The tank capacity, flow rates, sample-age limit, and serial processing limits are proposed settings for these tests. I will save them in the test files and record any changes before the final runs.

Control decisions and file input

C01 Thresholds and invalid settings

The control tests use the original 15 cases and 54 added cases in **boundaries.csv**. The added cases check levels 0, 2999, 3000, 3001, 5000, 6999, 7000, 7001, and 10000 with the valve starting open and closed. Below 3000 it must open. Above 7000 it must close. At either threshold and between them, it must keep its previous state. This checks that the valve does not switch just because the reading reaches a threshold.

The same file tests readings of 10001 and 65535, an invalid sample flag, reversed or equal thresholds, and a high threshold above 10000. Each invalid case must close the valve and set the fault flag. I also test valid settings of (0,10000), (0,1), and (9999,10000) to check the endpoints and a gap of one unit. The parser must reject 65536 before converting it to uint16_t. The value 65535 fits that type, but it is still outside the tank level range.

C02 Keeping state and recovering from faults

The 18-row **sequence.csv** keeps the valve and fault state from one step to the next. A reset field starts a new sequence, and the first row must reset. Starting closed, levels 2500, 3000, 5000, 7000, and 7001 must produce open, open, open, open, and closed. Returning through 7000, 5000, 3000, and 2999 must produce closed, closed, closed, and open.

The next rows supply an invalid sample and invalid settings, then restore valid data. The valve must close during a fault. When a valid reading of 5000 returns, the fault must clear and the valve must stay closed. A reading of 2500 must open it. I check both flags on every row because the final state alone could hide a wrong decision earlier in the sequence.

P01 Reading valid and invalid files

The 23 parser test files include 18 that should be rejected. They cover missing or extra fields, empty files, comments without data, negative or nonnumeric values, a leading plus sign or space, invalid Boolean fields, numeric overflow, trailing data, embedded NUL, Ctrl-Z, and oversized records. Each rejected file must return status 2 without crashing. One file deliberately gives the wrong expected valve state. It must return status 1 so I know the runner catches an incorrect result.

Four valid files cover LF, CRLF with a blank line, no final newline, and a 255-byte record including its newline. The 256-byte record must fail. The host reader has a 256-byte buffer with one byte reserved for the terminator. Files are opened in binary mode so Windows does not hide Ctrl-Z or translate bytes before validation. Both runners skip LF and CRLF blank lines consistently.

I also check a missing file, no filename, and an extra argument. Each must return status 2. The parser must check the entire number and its range before converting it to a smaller type. These tests cover the integer, string, array, and input-handling issues described by CERT C [2].

Changing tank and sensor tests

T01 through T06 Tank behavior

I will use **tank_runs.json** and **tank_events.csv** for the changing tank tests. At each timestamp, the test program will apply any scheduled changes, supply the sensor reading, run one control step, and read the valve output. It will then calculate the volume for the next step:

```
V_next = V + (actual_valve_open * inlet_Lps - demand_Lps) * dt_seconds
```

The model will keep fractional volume values and report floor(10000 * V / 100) as the level. It will record any calculated volume below 0 L or above 100 L before limiting the displayed volume to the tank capacity. This makes an empty or overflowing tank visible in the results. I will record the commanded valve state separately from the actual state in the model.

T01: I will run the default 600-second scenario three times. The tank gains or loses 0.1 L per step, so I expect it to cycle between about 29.9% and 70.1%. I will allow 2980 to 7020 basis points to account for the discrete steps and rounded sensor value. Starting at 50%, the tank must stay in that range, switch the valve repeatedly, and have no faults or overflow. Each valve decision must match the expected result calculated separately.

T02: I will change demand from 1 to 0.5 L/s at 200 seconds, then to 1.5 L/s at 400 seconds. Inlet flow will stay at 2 L/s. The level must stay between 2980 and 7020 basis points. This checks that changing demand affects how long the tank takes to fill and drain without changing the control rule.

T03: I will set demand to 3 L/s for 600 seconds. The tank will drain even with the inlet open because more water is leaving than entering. I expect the open command and an empty-tank event in the log. **T04:** I will force the modeled valve to stay open at 10 seconds in a 120-second run. The controller must command it closed above 70%, while the model records continued filling and overflow. These cases show the limits of the controller. It cannot supply missing water or close a stuck valve, and it does not yet detect actuator faults.

T05: I will run for 3600 seconds at 100 ms and check all 36001 samples, including time zero. **T06:** I will repeat T01 at 50, 100, and 500 ms. The allowed level range will be [3000-B, 7000+B], where B = ceil(200 * dt_seconds) + 1 basis points. The 200 bp/s limit covers the inlet and demand rates used here. The extra point allows for rounding. A longer sampling interval gives the level more time to move past a threshold before the valve responds.

S01 through S03 Sensor faults

I will use **sensor_events.csv** to supply an invalid flag, readings outside the allowed range, and an old sample. I plan to allow a sample age of up to 300 ms. A host test will check that age 300 is valid and age 301 is stale. With a 100 ms loop and the last fresh sample at time zero, the first scheduled stale-sample fault should occur at 400 ms. The valve must close on the first control step that detects the fault and return to normal control when valid data returns.

I will also reset the controller while the tank is filling and check that the output closes before the first control decision after reset. These checks cannot identify a false reading that still looks valid and recent. I need to finish the firmware changes before I can run the age, reset-timing, and tank feedback tests.

Serial commands and authentication

A01 Changing settings

I plan to use an ASCII command in the form **SET,low,high,tag**, ending with LF. The decimal fields will have no sign or leading zeros, except for the value 0. The tag will contain exactly 64 lowercase hexadecimal characters. HMAC-SHA256 will cover the exact bytes **SET,low,high**, leaving out the comma before the tag and the LF. I will finalize this format in the comprehensive design. The parser is still planned work.

The valid test command changes the thresholds from (3000,7000) to (3500,6500). I will check that both settings change together after the command passes validation. At level 3200 with the valve closed, the next control step must open it using the new low threshold. After a rejected command, both settings must remain unchanged. I will run another control step to check that the controller still works normally.

A02 Checking tags and values

I will first check the chosen library against HMAC-SHA256 test cases 1 and 2 from RFC 4231 [3]. The expected tags are supplied separately from the firmware. I will then send a valid command, change its contents without updating the tag, use a tag made with the wrong key, and send missing, short, long, or nonhex tags. Only a command with a valid tag and correct format may change the settings.

A command with reversed thresholds must be rejected even if its authentication tag is valid. I will also test equal thresholds, a high value above 10000, and the authenticated command containing 65536. The parser must check each number before converting it to a smaller type and validate both thresholds before changing either one. I plan to use an established library such as Mbed TLS and follow its guidance for checking tags. These tests check the results, but do not prove that tag comparisons take constant time.

A03 Incomplete and oversized commands

I plan to limit each line to 128 bytes excluding LF, with room for a terminator, and allow 500 ms to finish a partial line. I will send a valid command in one-byte and seven-byte pieces, send two commands together, and test NUL, CR, unknown commands, and lines of 127, 128, 129, and 1024 bytes. The supplied length-test files contain only X characters, so all are invalid commands. The 129-byte and 1024-byte lines also exceed the limit. The parser must reject a bad line instead of accepting a shortened version of it.

After a line exceeds the limit, the parser must discard input through the next LF. After a timeout, it must clear the partial command and accept the next complete valid line. I will test ages of 500 and 501 ms, with the timeout taking effect above 500 ms. Invalid input and timeouts must leave the settings unchanged and let level control continue.

A04 Replayed commands

The replay file sends command A, a different valid command B, then A again. There is no nonce or counter, so the controller may accept A again and restore the older settings. I will repeat this after reset and record the result. The project does not include replay protection. HMAC authenticates commands but does not encrypt them. The supplied key is public test data. As in the preliminary design, I am treating the host and emulator as trusted.

Running the integration tests

How I will check the results

The host CSV files contain the expected valve and fault states. For the tank tests, I will calculate the expected results separately using the volume equation and threshold rules. Using the same C function to produce both results could repeat the same mistake. I will compare every sample, settings change, and fault transition so a correct final volume cannot hide an earlier error.

The planned test program will read the scenario file, advance Renode time, supply the sensor reading and its validity and timestamp, and inspect PD12 for the valve output. I will try the ADC interface first. If it cannot accept the test readings, I will use a documented emulated sensor register. That would let me test the control loop, but I would still need a separate test for ADC conversion.

L01 Keeping control running during heavy input

I plan to process at most 32 received bytes and one authentication attempt in each 100 ms cycle, using a 256-byte receive queue. Each cycle will run the control step first. I will compare three 60-second runs: no commands, one valid command per second, and 1000 bytes/s of malformed input from seconds 20 through 30. The flood file contains 10000 bytes, which the test program will send over those 10 simulated seconds.

During the flood, I will mark the sensor reading invalid at 25 seconds. The valve must close at the next scheduled control step. The delay from a sample to its output must stay within 100 ms, and the controller must not skip scheduled steps. It must report a full receive queue, discard invalid lines, and accept a valid command after the flood ends. I will measure this in Renode simulated time. I may need to adjust these processing limits after I add the authentication library.

How I will run and record the tests

1. I will build from a clean extracted folder and record the operating system, Python and compiler versions, compiler options, firmware hash, Renode and library versions, and input hashes. I will run the host tests before starting the emulator.

2. I will reset the emulator for each separate scenario, load the default settings, and start with the valve closed. The test program will read the inputs from the selected files and keep state between steps unless a row requests a reset.

3. I will log the time, actual volume, sensor level, sample age and validity, thresholds, commanded and actual valve states, fault flag, command result, and queue-overflow count. For a failure, I will save the first mismatch and the surrounding rows.

4. I will repeat each normal scenario three times from reset and compare the recorded values, excluding host timestamps. This checks that the same input produces the same result. I will keep each run in a separate folder. After a fix, I will rerun the affected tests and the control and parser tests.

What counts as a pass

A host test passes when its outputs and exit code match the expected values. The integration tests must also meet the valve, timing, and settings checks described above. I will mark tests that cannot run as **not run**. When sanitizer tools are available, I will use them to check for memory errors and undefined behavior [4] and record any findings or tool failures. Testing on a physical device would still be needed to check electrical behavior and real device timing.

Results so far and running on other systems

What I ran for this submission

On November 9, I built the host programs on Windows with MinGW-w64 GCC 16.1.0, C17, -O1, and debug symbols. Compiler warnings were treated as errors. All 15 original cases, 54 added boundary cases, 18 sequence steps, 23 parser test files, and three command-line error checks passed. The deliberately wrong result returned 1, and malformed input returned 2. The logs are in **Evidence/Windows_Host**.

I also tried an undefined-behavior and bounds sanitizer build. The linker could not find libubsan, so that build did not run. I kept the error in **Evidence/Windows_Sanitizer_Attempt**. The earlier macOS tests passed 15 original cases, rejected eight files, and passed one fixed-reading Renode test. I have not run the expanded tests on macOS or Linux yet.

Running on Windows macOS and Linux

All three systems use **Scripts/run_tests.py** with Python 3.9 or newer and a C17 GCC or Clang compiler. The script finds files relative to its own location, builds for the local x86-64 or ARM64 host, and uses the same test data on each system. The host tests need only Python and the C compiler. They do not need Bash, WSL, Renode, ARM tools, or extra Python packages.

Windows: The expanded host tests passed on this x86-64 computer. ARM64 host runs remain to be checked.

```
py -3 Scripts/run_tests.py --cc gcc
```

macOS and Linux: The same script is ready to run on both systems. The older macOS tests passed, but I have not run the expanded suite on either system yet.

```
python3 Scripts/run_tests.py --cc cc
```

If the compiler is outside PATH, pass its full path in quotes after --cc. Use python in place of py or python3 if that is the installed command. I still need to run the expanded tests on macOS and Linux. I will compare the test results across systems, since compiler messages and file paths will differ.

Tools for the firmware tests

Scripts/build_firmware.py uses GNU Arm Embedded GCC to build the Cortex-M4 ELF. It needs the version of that toolchain for the host operating system. The included Renode test checks boot messages and a fixed low reading. Renode 1.17 Robot tests also need Python dependencies. On Windows they use Cygwin, or Linux under WSL; on macOS and Linux they use the installed renode-test launcher [5]. Those tools must be set up separately from the host tests.

I have not rerun the ARM or Renode tests on this Windows computer. I kept the original project separately. The controller and firmware in this submission are unchanged. I updated the host reader to use binary input and accept CRLF blank lines, and added a runner that keeps state between CSV rows.

How to run the tests

Run the supplied files

Extract the complete ZIP and start in its top-level folder. Run the host command for your system on the previous page. Each run creates a new **Runs** folder containing builds, stdout/stderr logs, and **summary.json**. The recorded Evidence folder is never overwritten. A failed build, timeout, unexpected exit code, or wrong output makes the runner exit nonzero.

```
python3 Scripts/run_tests.py --sanitize undefined
python3 Scripts/run_tests.py --sanitize address
python3 Scripts/build_firmware.py
renode-test -P 0 Source/tests/c-controller.robot
```

These are optional commands: sanitizer modes need compiler runtime support, firmware needs arm-none-eabi-gcc, and the final command needs a configured Renode testing environment. On Windows use py -3 for the Python commands and run renode-test from its configured Cygwin/WSL environment. Install dependencies separately; the scripts do not download or install software.

Files included

Source contains the controller, two host runners, firmware/startup/linker files, and the original Renode test. **Test_Data/Control** contains independent and stateful cases. **Test_Data/Parser** contains exact-byte inputs and a manifest of expected exit codes. **Test_Data/Planned** contains the future tank settings/events, sensor events, serial bytes, and HMAC fixtures; the host suite does not count those as executed tests.

```
Control: level_bp,valid,low,high,previous_open,expected_open,expected_fault
Sequence: reset,level_bp,valid,low,high,start_open,expected_open,expected_fault
```

Boolean fields are 0 or 1. With reset=1 the sequence uses start_open and clears fault before the step; with reset=0 it retains the preceding actual state and ignores start_open. Rows beginning with # are comments. Sensor event times are milliseconds, flow rates are liters/second, and tank volume is liters. Each sensor test ID starts a separate closed-valve/default-configuration run; S02 retains its sample between listed times.

Scripts/generate_test_data.py recreates the test files. The supplied files are ready to use. **Scripts/build_report.py** contains the editable report text and uses ReportLab to make the PDF. **SHA256SUMS.txt** lists hashes for the delivered files.

References

[1] Renode. Testing with Renode. https://renode.readthedocs.io/en/latest/introduction/testing.html

[2] SEI CERT C Coding Standard. Integer, array, string, and input/output guidance. https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard/

[3] M. Nystrom. RFC 4231, HMAC test vectors, test cases 1 and 2. December 2005. https://www.rfc-editor.org/rfc/rfc4231.html

[4] GNU GCC manual. Instrumentation Options. https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html

[5] Renode 1.17.0 README. Installation and Robot Framework prerequisites. https://github.com/renode/renode/blob/v1.17.0/README.md

Project baseline: my October 29 preliminary design and its C source. External references checked November 9, 2026.